



Informatics II

Exercise 6

Grgic Anto

Schedule

Date	Topic
27. Feb	C environment
06. Mar	Ex1(Introduction to C, basic sorting)
13. Mar	Ex2 (Recursion)
20. Mar	Ex3 (Complexity, Correctness)
27. Mar	Ex4 (Divide and Conquer, Recurrence)
03. Apr	NO LAB (Please attend another class) – Spring break
17. Apr	Ex6 (Pointers, Linked Lists)
24. Apr	Ex7(ADTs, Stacks, Queues)
01. May	NO LAB (Please attend another class) – Labor day
08. May	Ex9 (Hash Tables)
15. May	Ex10 (Dynamic Programming)
22. May	Ex11 (Graphs)
29. May	Recap
04. Jun	Exam at Messe 16:00-19:00

Pointers →

- To understand pointers and their utility it is important to understand how memory works in C
- A memory cell is composed of **ADDRESS** and **VALUE**
- The Address is the physical place in the RAM (or CPU) where the value gets saved, the value is it's content

address	content
<i>1af783</i>	22
<i>1af784</i>	<i>1af78a</i>
<i>1af785</i>	33
<i>1af786</i>	NIL
<i>1af787</i>	<i>1af785</i>
<i>1af788</i>	—
<i>1af789</i>	—
<i>1af78a</i>	44

***Example from the lecture. On the left there is the address (expressed in hexadecimal base) and on the right different kind of values. This model is the one you are expected to know for the exam, but it is a simplification of the reality.**

A more correct (but more complex) model

- Each address is a hexadecimal number between 0 and $2^{64} - 1$ (for 64 bit system).
- The Address doesn't occupy any memory, it uses the same concept of the algorithm to find prime number, where one creates an array of length N and if you want to look up if 50 is prime, you look the value at position 50
- Each "Value" cell has size of 1 byte (8 bit) and its itself a hexadecimal number.
- An integer a (4 Bytes) occupies 4 cells
- How does a program know how many cells to read?
When you declare a variable (Or when you pass it to a function) you always have to specify it's type. So the program at runtime always knows the type of the variable and knows how many cells to read and how to interpret/convert the number it gets (is it a integer, long, char, array of int, ecc)
- A cell can never be empty!! This is why programs that recovery data exists and can possibly recover deleted images, texts ecc.
A simple delete operation just "Forgets" the address where data gets stored, but the data stays there until it gets overwritten by new data.

Address	Byte value
0x7ffd1a2b	0x00
0x7ffd1a2c	0x00
0x7ffd1a2d	0x00
0x7ffd1a2e	0x2A ← this is 42

*0x preceding a number means "in hexadecimal base.

0b preceding a number mean binary.

Passing a variable by value VS by reference

- When passing variable in any language one has two options. Passing by value or by reference.
- Passing by value means that the “Value cell” gets copied and that copy is returned.
- Passing by reference means that the “Adress cell” gets copied and that copy is returned.
- Python always passes by reference (If you want to pass by value you need to manually copy a variable and pass the copy by reference)
- C always passes by value (If you want to pass by reference you need to pass the address)
 - This is where pointers come into play

Pointers →

- Pointers are 8 Bytes long positive numbers
- Why are pointer useful? Because C always passes variable by value. Pointers lets you functionally pass a variable by reference.
- Syntax:
 - * Has double meaning:
 - if used in a declaration it means “This is a pointer”
 - if used in an expression it means “Give me the value this pointer is addressing”
 - & Can only be used in expressions and it means “Give me the address of this variable”

```
int a = 10;

int *b = NULL;    //b is a pointer pointing to nothing

b = &a;           //b is a pointer adresssing a

*b = 5;           //the value b is adresssing is equal to 5 --> a is now equal to 5
```

Since a is an integer (4 Bytes), it occupies 4 cells (1 Byte each).
&a is the address of the first cell (1 Byte). But *&a will give you a 4 bytes number.
That is because at runtime C knows a is an integer, and * operator will look all appropriate cells and combine them into one value

Arrays and Pointers

- With the new model, arrays become much more evident.
- When getting an address of an array (just like a integer or a char) we are getting simply the value in the first cell.
- If I call `arr[4]` and my array is of type `int`, then C will actually compute `*(&arr[0] + 4)`. Those two expressions are equivalent in C. → **Here we are not summing `4 * sizeof(int)` because C does that automatically**
- But when an array is defined in `main`, the compiler has information about type and size, so if I try to call the 10th element of an array length 5 it will complain.

- An integer array of 5 elements will have

$$\frac{4 \frac{\text{Bytes}}{\text{integer}} * 5 \text{ integers}}{1 \frac{\text{Bytes}}{\text{Cell}}} = 20 \text{ Cell} \quad \text{in memory}$$

- When passing an array into a function C always decays the array to a pointer, pointing at the first element of the array. Then when defining the array in the function one has to specify the type.
 - This is why calling the 10th element of an array of length 5 is possible when done in a function. The compiler won't complain and just access the cell memory it thinks contains that value
- An array pointing to `NULL` is simply pointing to address 0, which by definition is invalid.

Malloc and free

- When defining any variable, C automatically hides the memory cells where it saved the value so no other program can overwrite it. But if I am directly working with memory, how can I guarantee that it won't happen? → **Malloc**
- **Malloc** is a function in C that takes as input a number **N** and returns a pointer to a cell. This pointer points to a cell that has two properties:
 - 1. the next **N** cells (in order) are free (not used by other programs).
 - 2. the next **N** cells's address will be hidden and other programs won't be able to access them.
- When a program has done its job, C automatically sets the cells it defined with variable to be free again, so other program can find and use them again. For memory cell you allocated with **Malloc**, you need to do that manually. → **free**
- **Free** is a function that takes as input a variable and returns nothing.
 - The memory cell holding the variable's value will be visible again and other programs will have access to them.
- If the memory doesn't get freed it will not be available to other programs until the ram gets reset (most commonly with a computer restart)

Structs and pointers

- A struct in C is a simple way to store data, they allow you to create your own variable type and use it.
- If you have a pointer to a struct and you want to access one of its fields you would have to do `*(struct-pointer).field`. C has a notation to type this faster: `struct -> next`. This is especially useful when doing something like: `(**(*node).next).next.key` or much more clear with the arrow notation: `node->next->next->key`

Linked lists

- A linked list is a list of nodes.
- Each node has at least two fields:
 - The value associated to the node
 - A pointer to the next node
- There are many types of linked lists with different properties
 - Linked list
 - Doubly linked list
 - Circular linked lists
 - Doubly circular linked list
 - Etc.



Guard/sentinel/dummy

- A guard is useful when passing a pointer in a function and wanting to modify it
- A guard ensures that the modification of the pointer is propagated outside of the function's scope

```
// Without guard
void delete(struct node* head) {
    head = head->next; // lost when function returns
}

// With guard (double pointer)
void delete(struct node** head) {
    *head = (*head)->next; // actually modifies the caller's pointer
}
```

Exercises:

Task1: Getting familiar with pointer syntax

Task2: Starting using pointers

Task3: More pointers and structs

Thank you for your attention.

**If you have any questions write in the Olat forum
or come up to me.**